Höhere technische Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs Bundesschulzentrum

Ascenta - Guiding your Vision

Diplomarbeit

Schulautonomer Schwerpunkt
Coding & AI

ausgeführt im Schuljahr 2025/2026 von:

Liam Vendel, 5AHELS

Manuel Zauner, 5AHELS

Fabian Lindlbauer, 5AHELS

Betreuer:

Dipl.-Ing. Mario Preishuber

Braunau am Inn, am 11.02.2026

Eidesstattliche Erklärung

Es wird an Eides statt erklärt, dass die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als angegebene Quellen und Hilfsmittel nicht direkt benutzt und die benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht wurden.

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Liam Vendel

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Manuel Zauner

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Fabian Lindlbauer

Abstract

This diploma thesis details the development of “Ascenta”, an intelligent assistive wearable designed to support visually impaired individuals. Given the challenges of navigating complex urban environments, the project aims to create an affordable solution using artificial intelligence to provide information about the user’s social surroundings.

The technical framework utilizes the Arduino Nicla Vision microcontroller. The firmware implements the FOMO architecture for efficient on-device face detection. For user interaction, a local and privacy-oriented voice recognition system was developed using the **Vosk engine** within a custom Android application. Communication is established via an optimized UDP protocol to ensure low-latency data streaming between the wearable and the mobile device.

The current implementation successfully recognizes faces within the field of view and determines proximity using Time-of-Flight sensors. The results demonstrate the efficiency of the decentralized architecture and provide a solid foundation for future development of general obstacle detection models.

Kurzfassung

Die vorliegende Diplomarbeit dokumentiert die Entwicklung von „Ascenta“, einer intelligenten Assistenzbrille zur Unterstützung sehbeeinträchtigter Personen im Alltag. Angesichts der Komplexität moderner urbaner Umgebungen besteht das Ziel des Projekts in der Realisierung eines kostengünstigen Wearables, das mittels künstlicher Intelligenz Informationen über das soziale Umfeld liefert.

Die technische Umsetzung basiert auf dem Arduino Nicla Vision Mikrocontroller. Die Firmware nutzt die FOMO-Architektur (Faster Objects, More Objects) zur effizienten On-Device-Gesichtserkennung. Zur Interaktion wurde eine lokale, datenschutzkonforme Sprachverarbeitung auf Basis der **Vosk-Engine** in einer Android-Applikation implementiert. Die Kommunikation erfolgt über ein optimiertes UDP-Protokoll, um minimale Latenzzeiten beim Datenstreaming zwischen der Brille und dem Smartphone zu gewährleisten.

Das realisierte System ist in der Lage, Personen im Sichtfeld zu identifizieren und die Distanz mittels Time-of-Flight-Sensorik zu bestimmen. Die Ergebnisse belegen die Funktionalität der dezentralen Architektur und bilden die Grundlage für zukünftige Erweiterungen zur allgemeinen Hinderniserkennung.

Contents

Abstract iv

Kurzfassung v

1 Einleitung 2
- 1.1 Motivation und Ausgangssituation . . . 2
- 1.2 Problemstellung . . . 2
- 1.3 Zielsetzung . . . 2
- 1.4 Lösungsansatz und Kernidee . . . 3

2 Szenarien und Use Cases 4
- 2.1 Use Case A: Kollisionsvermeidung mit Personen . . . 4
  - 2.1.1 Ausgangssituation . . . 4
  - 2.1.2 Systemverhalten . . . 4
- 2.2 Use Case B: Kontextbezogene Umgebungsabfrage per Sprachbefehl . . . 5
  - 2.2.1 Ausgangssituation . . . 5
  - 2.2.2 Systemverhalten . . . 5

3 Konzept 6
- 3.1 Gesamtüberblick . . . 6
- 3.2 Komponenten . . . 6
  - 3.2.1 Mechanik und Gehäusedesign . . . 6
  - 3.2.2 Hardware und Elektronik . . . 6
  - 3.2.3 Software-Ökosystem . . . 7
- 3.3 Interaktionen . . . 7
- 3.4 Technisches Design . . . 7

4 Umsetzung 8
- 4.1 Implementierung der Komponenten . . . 8
  - 4.1.1 Konstruktion und 3D-Druck . . . 8
  - 4.1.2 Schaltungsdesign und Energiemanagement . . . 8
  - 4.1.3 Entwicklung des Machine-Learning-Modells (Objekterkennung) . . . 8
  - 4.1.4 Software-Architektur und Datenverarbeitung . . . 8

4.2 Herausforderungen in der Entwicklung . . . 18
4.2.1 Mechanische und geometrische Restriktionen . . . 18
4.2.2 Energiemanagement im begrenzten Bauraum . . . 18
4.2.3 Ressourcenlimits und Betriebssystem-Restriktionen (Software) . . . 18
4.3 Tests und Validierung . . . 19

5 Ausblick (Future Work) 20
5.1 Software und Benutzeroberfläche . . . 20
5.1.1 Native Zero-UI-Applikation für blinde Nutzer . . . 20
5.1.2 Kontinuierliches auditives Raum-Feedback (Leitstrahl-Prinzip) . . . 20
5.2 Künstliche Intelligenz und Modell-Training . . . 21
5.2.1 Erweiterte Objekterkennung für Alltagsgegenstände . . . 21
5.2.2 Weiterentwicklung des Machine-Learning-Modells (Edge Impulse) . . . 21
5.3 Hardware und Elektronik . . . 21
5.3.1 Optimierung der Schaltungstechnik und Sensorik . . . 21
5.4 Mechanik und Gehäusedesign . . . 21
5.4.1 Weiterentwicklung des 3D-Modells und der Ergonomie . . . 21

6 Verwandte Arbeiten (Related Work) 22
6.1 Klassische Mobilitätshilfen . . . 22
6.2 Smartphone-basierte Applikationen . . . 22
6.3 Kommerzielle Smart Glasses und Wearables . . . 23
6.4 Positionierung des Ascenta-Projekts . . . 23

7 Conclusion 24

8 Project Management 25
8.1 Planning . . . 25
8.2 Evaluation . . . 25
8.3 Timesheet . . . 26
8.3.1 Liam Vendel . . . 26
8.3.2 Manuel Zauner . . . 26
8.3.3 Fabian Lindlbauer . . . 26

Acknowledgements 27

List of Figures 27

Listings 28

Bibliography 29

CV 30

1 Einleitung

1.1 Motivation und Ausgangssituation

Stellen Sie sich vor, Sie navigieren während der Stoßzeit durch einen belebten Bahnhof. Sie verlassen sich auf Schilder, um den Ausgang zu finden, prüfen Abfahrtstafeln und achten auf andere Pendler. Für weltweit etwa 285 Millionen sehbehinderte Menschen stellt dieses Szenario eine tägliche Herausforderung voller Barrieren dar. Während der klassische Langstock Hindernisse am Boden erkennt, versagt er bei der Identifikation von Personen oder Objekten auf Kopfhöhe. Bestehende elektronische Hilfsmittel sind oft aufgrund extrem hoher Kosten ($>$ 2.500 Euro) oder einer Abhängigkeit von permanenter Cloud-Konnektivität für die breite Masse unzugänglich.

1.2 Problemstellung

Das zentrale Problem liegt in der mangelnden digitalen Inklusion aufgrund finanzieller und technologischer Hürden. Mobile Assistenzsysteme müssen eine Echtzeit-Verarbeitung sensibler Bild- und Audiodaten ermöglichen, ohne die Ergonomie des Nutzers durch hohes Gewicht oder Wärmeentwicklung zu beeinträchtigen. Zudem erfordern Cloud-basierte Ansätze eine ständige Internetverbindung, was die Zuverlässigkeit in geschlossenen Räumen (z. B. U-Bahnhöfen) mindert und den Datenschutz gefährdet.

1.3 Zielsetzung

Das Ziel des Projekts „Ascenta“ ist die Entwicklung eines Prototyps einer smarten Assistenzbrille, die visuelle Umweltinformationen in akustisches Feedback übersetzt. Das System soll kostengünstig (Materialkosten $<$ 150 Euro) realisiert werden, eine diskrete Bauweise aufweisen und durch lokale Datenverarbeitung (Offline-Betrieb) maximale Datensicherheit für den Nutzer garantieren.

1.4 Lösungsansatz und Kernidee

Die Lösung basiert auf einer dezentralen „Split-Brain“-Architektur. Die Brille (Arduino Nicla Vision) fungiert als smarter Sensor-Satellit und führt eine ressourcensparende Objekterkennung (FOMO) zur Gesichtserkennung aus. Die rechenintensive Logik sowie die Spracherkennung werden auf das Smartphone des Nutzers ausgelagert. Die Übertragung erfolgt über ein latenzarmes WiFi-UDP-Protokoll. Durch diesen Ansatz wird die Hardware an der Brille entlastet, während die volle Prozessorleistung moderner Smartphones für die Sprachverarbeitung mittels Vosk genutzt wird.

2 Szenarien und Use Cases

Um die funktionalen Anforderungen an das Assistenzsystem „Ascenta“ zu definieren und die spätere technische Umsetzung zu rechtfertigen, werden im Folgenden konkrete Anwendungsszenarien (Use Cases) beschrieben. Diese Szenarien repräsentieren alltägliche Herausforderungen von stark sehbeeinträchtigten oder blinden Personen, die durch das System gelöst werden. Gemäß dem aktuellen Entwicklungsstand fokussieren sich die Anwendungsfälle auf die Erkennung von Personen im direkten Umfeld, die Distanzabschätzung sowie die auditive Interaktion.

2.1 Use Case A: Kollisionsvermeidung mit Personen

2.1.1 Ausgangssituation

Ein Nutzer bewegt sich mit einem regulären Langstock durch einen öffentlichen Raum, beispielsweise ein Foyer oder einen Gehweg. Während der Langstock Bodenhindernisse (wie Treppenstufen oder Bordsteinkanten) zuverlässig ertastet, registriert er keine stillstehenden oder sich nähernden Personen auf Körper- oder Kopfhöhe, bevor es zu einem physischen Kontakt kommt.

2.1.2 Systemverhalten

Das Kamerasystem der Ascenta-Brille erfasst kontinuierlich das Sichtfeld des Nutzers. Sobald eine Person in den Erfassungsbereich tritt, wird diese durch das lokale Machine-Learning-Modell (FOMO) auf dem Mikrocontroller identifiziert. Simultan misst der Time-of-Flight-Sensor (ToF) die exakte Distanz zu diesem Objekt.

Unterschreitet die Person eine vordefinierte kritische Distanz (beispielsweise einen Meter), fusioniert die Smartphone-Applikation diese Sensordaten und generiert unmittelbar ein akustisches Warnsignal. Der Nutzer wird dadurch in Echtzeit auf das Hindernis aufmerksam gemacht und kann rechtzeitig ausweichen, ohne dass ein physischer Kontakt entsteht.

2.2 Use Case B: Kontextbezogene Umgebungsabfrage per Sprachbefehl

2.2.1 Ausgangssituation

Der Nutzer befindet sich in einer Warteschlange oder an einem Haltepunkt (z. B. einer Bushaltestelle) und möchte wissen, ob sich eine Person direkt vor ihm befindet, um entsprechend aufrücken oder Abstand halten zu können. Ein weites Ausschwenken des Langstocks ist in dieser Situation unerwünscht, da es andere Personen stören oder irritieren könnte.

2.2.2 Systemverhalten

Anstatt die Umgebung physisch abzutasten, nutzt der Anwender die integrierte Sprachsteuerung. Der Nutzer aktiviert das System (beispielsweise durch ein definiertes Interaktionsmuster an der Brille) und stellt verbal eine Frage, wie etwa: „Befindet sich jemand vor mir?“.

Das Audiosignal wird über das in der Brille verbaute Mikrofon erfasst und via UDP-Protokoll latenzarm an das verbundene Smartphone gestreamt. Dort verarbeitet die lokale Vosk-Engine den Sprachbefehl offline, um den Datenschutz zu gewährleisten. Die Applikation gleicht die Anfrage mit den aktuellen Kameradaten (Personenerkennung) und der Distanzmessung (ToF) ab. Anschließend erhält der Nutzer über die Sprachausgabe (Text-to-Speech) ein diskretes, akustisches Feedback, beispielsweise: „Person in einem Punkt fünf Metern Entfernung erkannt.“ Dadurch gewinnt der Nutzer ohne stigmatisierende Bewegungen ein präzises Verständnis seiner unmittelbaren sozialen Umgebung.

3 Konzept

Dieses Kapitel beschreibt die abstrakte Architektur des Assistenzsystems „Ascenta“. Es liefert einen Gesamtüberblick über das funktionale Zusammenwirken der Teilsysteme, ohne dabei auf tiefergehende Implementierungsdetails einzugehen.

3.1 Gesamtüberblick

Das System basiert auf einer dezentralen „Split-Brain“-Architektur. Um die Hardware direkt am Kopf des Nutzers so leicht und unauffällig wie möglich zu gestalten, wird die ressourcenintensive Datenverarbeitung auf das Smartphone des Nutzers ausgelagert. Die Brille fungiert als reiner Sensor- und Kommunikationsknotenpunkt (Edge-Device), der Umgebungsdaten erfasst und Ergebnisse der lokalen Vorverarbeitung an das mobile Endgerät streamt. Das Smartphone übernimmt die Rolle des Hauptrechenzentrums, welches die Sensordaten fusioniert, Benutzereingaben interpretiert und das akustische Feedback generiert.

3.2 Komponenten

Das Gesamtsystem setzt sich aus drei wesentlichen logischen Bausteinen zusammen: der physischen Struktur (Gehäuse), der elektronischen Hardware (Sensorik und Energieversorgung) und dem Software-Ökosystem.

3.2.1 Mechanik und Gehäusedesign

[Hier fügt Manuel seine konzeptionellen Gedanken zum Gehäusedesign und den ergonomischen Anforderungen ein (Warum 3D-Druck, Gewichtsverteilung, etc.).]

3.2.2 Hardware und Elektronik

[Hier fügt Fabian die konzeptionelle Begründung der Elektronik und Bauteilwahl ein (Warum Nicla Vision, warum LiPo-Akkus, grobes Energiekonzept).]

3.2.3 Software-Ökosystem

Die Softwarearchitektur teilt sich in die Firmware des Mikrocontrollers und die mobile Applikation. Die Firmware (MicroPython) ist für die hardwarenahe Sensorsteuerung und eine erste Datenreduktion (Edge AI mittels FOMO) verantwortlich. Die Android-Applikation (Kotlin) dient als unsichtbares Interface (Zero-UI) und integriert komplexe Algorithmen wie die Offline-Spracherkennung (Vosk) zur auditiven Interaktion.

3.3 Interaktionen

Die Interaktion zwischen den Systemkomponenten und dem Nutzer verläuft in einem kontinuierlichen Kreislauf:

1. **Datenerfassung:** Kamera und Time-of-Flight-Sensor erfassen synchron die visuelle Umgebung und Distanzdaten. Parallel überwacht die Inertial Measurement Unit (IMU) physische Eingaben durch den Nutzer.
2. **Vorverarbeitung und Übertragung:** Der Mikrocontroller identifiziert relevante Objektkoordinaten und überträgt diese, bei Bedarf ergänzt durch Audio-Fragmente, asynchron über ein lokales WiFi-Netzwerk an das Smartphone.
3. **Fusion und Auswertung:** Das Smartphone verknüpft die visuellen Koordinaten mit den Distanzwerten und wertet eventuelle Sprachbefehle aus.
4. **Feedback:** Überschreitet ein Objekt einen kritischen Schwellenwert oder erfordert ein Sprachbefehl eine Antwort, generiert das Smartphone ein auditives Signal oder eine Sprachausgabe (TTS), die der Nutzer wahrnimmt.

3.4 Technisches Design

Das technische Design der Kommunikation stellt sicher, dass kritische Warnungen höchste Priorität erhalten. Die Wahl des User Datagram Protocols (UDP) für Discovery-Prozesse und TCP für den asynchronen Binärdaten-Stream ist eine bewusste Designentscheidung. Zudem garantiert die konsequente Offline-Ausführung aller Machine-Learning-Modelle den Schutz sensibler Nutzerdaten (Privacy-by-Design), da keine Umgebungsvideos oder Audiodaten an externe Cloud-Anbieter übermittelt werden.

4 Umsetzung

Nachdem im vorherigen Kapitel das theoretische Konzept definiert wurde, beschreibt dieses Kapitel die praktische Realisierung des Prototyps.

4.1 Implementierung der Komponenten

4.1.1 Konstruktion und 3D-Druck

[Hier fügt Manuel seinen sachlichen Text zur CAD-Modellierung in Fusion 360, den Maßen, dem Klappmechanismus und dem Druckvorgang ein.]

4.1.2 Schaltungsdesign und Energiemanagement

[Hier fügt Fabian seine Schaltpläne, die genaue Verkabelung (TP4056, Nicla) und die Entladungskurven der Akkus ein.]

4.1.3 Entwicklung des Machine-Learning-Modells (Objekterkennung)

[Hier fügt Fabian seinen Text zum Training des Modells mit Roboflow, Kaggle und Edge Impulse ein.]

4.1.4 Software-Architektur und Datenverarbeitung

Die softwareseitige Umsetzung erfordert die Koordination eines ressourcenbeschränkten Embedded-Systems (MicroPython) und eines mobilen Betriebssystems (Android). Im Folgenden werden die Lösungsansätze für die Netzwerkkommunikation, die asynchrone Datenverarbeitung und das Speichermanagement detailliert erläutert.

Android Lifecycle-Management und Systementkopplung

Da das Smartphone während der Nutzung der Assistenzbrille typischerweise in der Tasche des Nutzers verbleibt und der Bildschirm deaktiviert ist, neigt das Android-Betriebssystem dazu, Hintergrundprozesse aus Energiespargründen zu terminieren (Doze-Mode). Um dies zu

verhindern, wurde die gesamte Netzwerk- und Bluetooth-Logik in einen Android `Service` (`AscentaService.kt`) ausgelagert.

Durch den Aufruf von `startForeground()` wird dem System höchste Priorität signalisiert. Ein weiteres kritisches Problem bestand darin, dass Android im Standby-Modus den Empfang von UDP-Broadcast-Paketen auf Hardware-Ebene blockiert. Dies wurde gelöst, indem ein `WifiManager.MulticastLock` akquiriert wird, was den WiFi-Chip zwingt, Broadcast-Pakete auch bei deaktiviertem Display durchzureichen:

```
val wifiManager = applicationContext.getSystemService(Context.WIFI_SERVICE) as
    WifiManager
multicastLock = wifiManager.createMulticastLock("AscentaUDP")
multicastLock?.setReferenceCounted(true)
multicastLock?.acquire()

startForeground(1, createNotification("Ascenta Active"))
```

Listing 4.1: Akquirierung des MulticastLocks und Start des Foreground Services

Hybrides Netzwerkprotokoll (BLE, UDP, TCP)

Die Kommunikation durchläuft einen dreistufigen Prozess („Zero-Config“), um manuelle IP-Konfigurationen zu vermeiden.

1. Phase: BLE Provisioning
Im unkonfigurierten Zustand öffnet das Nicla Vision einen BLE GATT-Server. Die Android-Applikation verbindet sich und schreibt die WLAN-Credentials in dedizierte BLE-Characteristics. Danach verbindet sich der Mikrocontroller mit dem WLAN.

2. Phase: UDP Discovery
Nach der WLAN-Einwahl sendet die Brille periodisch `NICLA_READY`-Broadcasts via UDP auf Port 5006. Eine Kotlin-Coroutine in der App lauscht asynchron auf diese Pakete, um die dynamisch vergebene IP-Adresse der Brille zu ermitteln:

```
udpSocket = DatagramSocket(UDP_DISC_PORT).apply { broadcast = true }
val packet = DatagramPacket(buffer, buffer.size)

while (isActive && !isConnected) {
    udpSocket?.receive(packet)
    val str = String(packet.data, 0, packet.length).trim()
    if (str == "NICLA_READY") {
        udpSocket?.close()
        connectTcp(packet.address) // TCP-Handshake einleiten
        break
    }
}
```

Listing 4.2: Asynchroner UDP-Discovery-Listener in Kotlin

3. Phase: TCP Datenstrom

Nach der IP-Ermittlung etabliert die App eine TCP-Verbindung (Port 5005). TCP verhindert Paketverluste bei der Übertragung großer binärer Payloads (WAV-Audio-Chunks). Auf dem Mikrocontroller wird der Socket mittels `conn.setblocking(False)` konfiguriert, damit Verbindungsabbrüche keinen Deadlock verursachen.

Ausfallsicherheit durch Heartbeat-Mechanismus

Um Verbindungsabbrüche zuverlässig zu erkennen, wurde in der `MainActivity` ein asynchroner Watchdog implementiert. Die Applikation sendet alle zehn Sekunden einen Ping an die Brille. Bleibt der Response für mehr als 25 Sekunden aus, wird die Verbindung sicher geschlossen und der UDP-Discovery-Modus reaktiviert (*Self-Healing*):

```
heartbeatJob = CoroutineScope(Dispatchers.IO).launch {
    while (isActive) {
        if (isConnected) {
            sendCommand("ping")
            delay(10000)
            if (System.currentTimeMillis() - lastPingResponse > 25000) {
                withContext(Dispatchers.Main) {
                    isConnected = false
                    stopHeartbeat()
                    showHome() // Zurueck in den Discovery-Status
                }
            }
        } else { delay(5000) }
    }
}
```

Listing 4.3: Heartbeat-Coroutine zur Überwachung der Verbindungsstabilität

Kinetische Interaktionssteuerung (Tap-Detection)

Zur Aktivierung der Spracherkennung wurde eine knopflose Bedienung implementiert. Der Beschleunigungssensor (LSM6DSOX) wird via SPI ausgelesen. Anstatt absolute Beschleunigungswerte zu nutzen, berechnet die Firmware den Ruck (Δg) zwischen zwei Frame-Messungen, um normale Kopfbewegungen herauszufiltern:

```
x, y, z = self.lsm.accel()
dx = abs(x - self.last_accel[0])
dy = abs(y - self.last_accel[1])
dz = abs(z - self.last_accel[2])
self.last_accel = (x, y, z)

delta_g = dx + dy + dz

# Delta > 1.75g entspricht einem harten physischen Tap
```

```
if delta_g > 1.75:
    now = time.ticks_ms()
    diff = time.ticks_diff(now, self.last_tap_time)

    if 100 < diff < 600:
        self.send_tcp_packet(b'{"action": "tap"}\n')
        self.req_audio = True # Audio-Streaming starten
        self.last_tap_time = 0
    elif diff > 600:
        self.last_tap_time = now
```

Listing 4.4: Berechnung von Delta-G zur Erkennung eines Double-Taps auf den Brillenrahmen

Speichereffizientes Audio-Streaming und Chunking

Eine Audioaufnahme übersteigt bei Weitem den RAM des Nicla Vision. Die Firmware nutzt einen *Circular Buffer* (`g_audio_chunks`), bei dem Audiodaten mittels `memoryview` in 1024-Byte-Chunks geschrieben werden, ohne neuen Speicher zu allozieren.

Auf dem Smartphone wird die resultierende WAV-Datei an die *Vosk Offline Speech Recognition Engine* übergeben. Um UI-Freezes zu verhindern, wird die Datei schrittweise in 4096-Byte-Blöcken eingelesen:

```
val chunkSize = 4096
var offset = 0
while (offset < audioData.size) {
    val length = minOf(chunkSize, audioData.size - offset)
    recognizer.acceptWaveForm(
        audioData.copyOfRange(offset, offset + length),
        length
    )
    offset += length
}
```

Listing 4.5: Speichereffizientes Chunking zur Vermeidung von Buffer-Overflows

Audio-Verarbeitung und kognitive Entlastung

Nach Auswertung der Sensordaten generiert das System Sprachwarnungen. Um den blinden Nutzer vor einer auditorischen Überlastung zu bewahren, implementiert die App eine dynamische Cooldown-Logik. Objekte in unkritischer Entfernung (über 50 cm) lösen eine Blockade von sechs Sekunden aus. Nähert sich ein Objekt jedoch in den kritischen Nahbereich (unter 50 cm), wird der Cooldown drastisch reduziert:

```
val distanceMm = dist.toIntOrNull() ?: 0
val isCollisionWarning = distanceMm in 10..500
```

```
val activeCooldown = if (isCollisionWarning) 3000L else SPEECH_COOLDOWN

if (now - lastSpokenTime > activeCooldown || label != lastSpokenLabel) {
    lastSpokenLabel = label
    lastSpokenTime = now

    if (isCollisionWarning) {
        speakText("Kollisionswarnung! $label direkt vor dir!")
    } else {
        // Regulaere Distanzansage
    }
}
```

Listing 4.6: Dynamische Cooldown-Logik in Abhängigkeit der Objektdistanz

3. Phase: TCP Datenstrom

Nach der IP-Ermittlung etabliert die App eine verbindungsorientierte TCP-Verbindung (Port 5005). TCP wurde hierbei dem UDP-Protokoll vorgezogen, um Paketverluste bei der Übertragung großer binärer Payloads (WAV-Audio-Chunks oder JPEG-Bilder) strikt zu verhindern. Auf dem Nicla Vision wird der Socket mittels `setblocking(False)` konfiguriert, damit abgebrochene Verbindungen den Mikrocontroller nicht in einen Deadlock versetzen.

Ausfallsicherheit durch Heartbeat-Mechanismus

Um Verbindungsabbrüche (z. B. durch Verlassen der WLAN-Reichweite) zuverlässig zu erkennen, wurde ein asynchroner Watchdog implementiert. Die Android-Applikation sendet in einer parallelen Coroutine alle zehn Sekunden einen Ping-Befehl an die Brille. Bleibt der entsprechende `pong`-Response des Mikrocontrollers für mehr als 25 Sekunden aus, deklariert die Software den Verbindungsstatus als unterbrochen, schließt die Sockets sicher (`try-catch` Blöcke) und kehrt automatisch in den UDP-Discovery-Modus zurück, um eine Re-Integration zu initiieren (*Self-Healing*).

Ressourcenmanagement und Modell-Integration (Nicla Vision)

Die Firmware des Nicla Vision (`Firmware.py`) operiert unter starken Speicherrestriktionen. Um Out-of-Memory (OOM) Exceptions während Laufzeit-Interrupts zu verhindern, reserviert das System initial einen Notfall-Puffer (`micropython.alloc_emergency_exception_buf(10`

Die Integration des trainierten FOMO-Modells (siehe Abschnitt 4.1.3) erfolgt ressourceneffizient direkt im Framebuffer des Mikrocontrollers (`load_to_fb=True`). Iteriert die Firmware über die Grid-Zellen der generierten Heatmap und verifiziert einen Konfidenzwert von über 45 %, wird parallel der VL53L1X Time-of-Flight-Sensor zur Distanzmessung über den I2C-

Bus ausgelesen. Diese aggregierten Daten werden als kompaktes JSON-Objekt an das Smartphone gestreamt.

Kinetische Interaktionssteuerung (Tap-Detection)

Zur Aktivierung der Spracherkennung wurde eine knopflose Bedienung implementiert, da physische Tasten die IP-Zertifizierung des Gehäuses beeinträchtigen würden. Hierbei wird der Beschleunigungssensor (LSM6DSOX) via SPI ausgelesen. Anstatt auf absolute Beschleunigungswerte zu reagieren, welche bei normalen Kopfbewegungen ebenfalls hoch ausfallen, berechnet der Algorithmus die zeitliche Änderung der Beschleunigung (Ruck bzw. Δg):

$$\Delta g = |x_t - x_{t-1}| + |y_t - y_{t-1}| + |z_t - z_{t-1}| \tag{4.1}$$

Übersteigt Δg einen Schwellenwert von $1,75\,g$, und erfolgt ein zweiter Impuls innerhalb eines Zeitfensters von 100 bis 600 ms, wird dies als beabsichtigter Doppeltipp (Double-Tap) gegen den Brillenrahmen interpretiert. Dieser Hardware-Interrupt blockiert kurzzeitig die Bildverarbeitung und startet unmittelbar das Audio-Streaming.

Speichereffizientes Audio-Streaming

Eine Audioaufnahme (2,5 s, 16 kHz, Mono) erzeugt Datenmengen, die den freien Arbeitsspeicher des Nicla Vision übersteigen würden, wenn sie am Stück gepuffert werden. Die Firmware implementiert daher einen *Circular Buffer*. Die Audiodaten werden während der Aufnahme über einen Callback (`_audio_callback`) iterativ in 1024-Byte-Chunks geschrieben, wobei ein `memoryview` verhindert, dass bei der Array-Zuweisung neuer Speicher alloziert werden muss. Die Firmware generiert dynamisch einen 44 Byte großen RIFF/WAVE-Header mittels `struct.pack` und streamt diesen, gefolgt von den sequenziellen Audio-Chunks, direkt über den TCP-Socket.

Audio-Verarbeitung und kognitive Entlastung

Auf dem Smartphone wird die WAV-Datei asynchron an die *Vosk Offline Speech Recognition Engine* übergeben. Um UI-Freezes zu verhindern, wird die Audiodatei nicht im Ganzen, sondern speichereffizient in 4096-Byte-Blöcken eingelesen.

Nach der Auswertung von Sensor- und Sprachdaten generiert das System Warnungen über die `TextToSpeech`-API. Um den blinden Nutzer jedoch vor einer Reizüberflutung (auditory overload) zu bewahren, implementiert die Applikation eine dynamische Cooldown-Logik in der Methode `onDetectionResult`: Wird ein Objekt in unkritischer Entfernung (über 500 mm) erkannt, blockiert das System weitere Ansagen desselben Objekts für sechs Sekunden. Tritt das Objekt jedoch in den Nahbereich (10 mm bis 500 mm) ein, wird dies als

kritische Kollisionsgefahr gewertet. Der Cooldown wird auf drei Sekunden reduziert und der Wortlaut der Warnung eskaliert auf ein direktives „Kollisionswarnung!“.

Auswertung des 4D-Tensors und Sensorfusion (Edge AI)

Eine besondere algorithmische Herausforderung bestand in der Auswertung der Vorhersagedaten (Inference) des FOMO-Modells. Im Gegensatz zu klassischen Objekterkennungsmodellen (wie YOLO), die fertige Bounding-Box-Koordinaten zurückliefern, generiert das quantisierte FOMO-Modell einen vierdimensionalen Tensor (Heatmap), der das Bild in ein Grid unterteilt.

Die Firmware auf dem Nicla Vision muss diesen Tensor in Echtzeit parsen. Hierfür iteriert der Algorithmus über das X/Y-Grid und die definierten Objektklassen. Um falsch-positive Erkennungen zu minimieren, wird ein strikter Konfidenz-Schwellenwert (Confidence-Threshold) von 45 % $(0,45)$ angewendet. Wird ein lokales Maximum gefunden, berechnet die Firmware den absoluten Bildschwerpunkt (Zentroid) und triggert synchron den Time-of-Flight-Sensor (VL53L1X), um den 2D-Bildkoordinaten eine physische Z-Achse (Distanz in Millimetern) hinzuzufügen:

```
# raw_output[0] enthaelt den 4D-Tensor: [Batch, Grid_Y, Grid_X, Classes]
grid_y, grid_x, classes = shape[1], shape[2], shape[3]
cell_w, cell_h = img.width() / grid_x, img.height() / grid_y

for y in range(grid_y):
    for x in range(grid_x):
        for c in range(1, classes):
            score = heatmap[0][y][x][c]
            if score > 0.45 and score > best_conf:
                best_conf = score
                best_label = self.labels[c]
                # Berechnung des Zentroids (Mittelpunkt der Zelle)
                best_x = int(x * cell_w + (cell_w / 2))
                best_y = int(y * cell_h + (cell_h / 2))

# Sensorfusion: Hinzufuegen der Z-Achse (Distanz)
if self.tof:
    distance_mm = self.tof.read()
```

Listing 4.7: Parsing der Heatmap (4D-Tensor) und Fusion mit ToF-Distanzdaten

Low-Level Hardware-Kommunikation (I2C & Registerauslesung)

Um dem Nutzer zuverlässige Batteriewarnungen geben zu können, reicht es bei LiPo-Akkus aufgrund ihrer flachen Entladekurve nicht aus, lediglich die anliegende Spannung zu messen. Daher wird der integrierte Fuel-Gauge-IC (MAX17262) über den I2C-Bus (Inter-Integrated Circuit) ausgelesen.

Diese Kommunikation erfordert Low-Level-Hardwarezugriffe. Der Mikrocontroller adressiert den Chip unter der Hexadezimal-Adresse `0x36` und fordert zwei Bytes ab dem Register `0x06` (RepSOC - Reported State of Charge) an. Da der Chip die Daten im Little-Endian-Format überträgt, müssen die rohen Bytes softwareseitig in einen Integer konvertiert und anschließend durch $256,0$ dividiert werden, um den exakten prozentualen Ladezustand (SoC) zu berechnen:

```
def get_soc(self):
    if not self.fuel_gauge_i2c: return None
    try:
        # Auslesen von 2 Bytes aus Register 0x06 (RepSOC)
        data = self.fuel_gauge_i2c.readfrom_mem(0x36, 0x06, 2)

        # Konvertierung von Little-Endian Bytes zu Integer
        soc_raw = int.from_bytes(data, 'little')

        # Umrechnung in Prozent gemaess Datenblatt
        soc = soc_raw / 256.0
        return soc
    except:
        return None
```

Listing 4.8: Register-Level I2C-Auslesung des MAX17262 Fuel-Gauge-ICs

Concurrency und Thread-Sicherheit in Kotlin

Da die Android-Applikation netzwerkbasierte I/O-Operationen (TCP/UDP-Sockets) und rechenintensive Machine-Learning-Prozesse (Vosk-Spracherkennung) simultan ausführen muss, war die Implementierung einer robusten Nebenläufigkeit (Concurrency) zwingend erforderlich. Ein blockierter Main-Thread (UI-Thread) würde unweigerlich zu einer *Application Not Responding* (ANR) Exception und somit zum Absturz führen.

In der Kotlin-Architektur wurde dieses Problem durch striktes Thread-Dispatching mittels *Coroutines* gelöst. Sämtliche Sockets und blockierende Lesevorgänge (`tcpSocket!!.getInputStream()`) operieren ausschließlich im `Dispatchers.IO` Thread-Pool. Sobald ein Datenpaket vollständig empfangen und geparst wurde (beispielsweise ein validiertes JSON-Objekt), erfolgt ein synchronisierter Kontextwechsel zurück auf den Main-Thread (`handler.post` oder `runOnUiThread`), um UI-Updates durchzuführen oder die `TextToSpeech`-Engine anzusteuern:

```
scope.launch(Dispatchers.IO) { // Start im Hintergrund-Thread
    val input = tcpSocket!!.getInputStream()
    while (isActive && isConnected) {
        val line = readLineFromStream(input) // Blockierender I/O-Aufruf

        if (line.startsWith("{")) {
```

```
            val json = JSONObject(line)
            val label = json.optString("label")

            // Sicherer Wechsel auf den Main-Thread fuer das UI/Audio-Feedback
            handler.post {
                callback?.onDetectionResult(label, conf, dist)
            }
        }
    }
}
```

Listing 4.9: Sicherer Kontextwechsel zwischen IO-Thread und Main-Thread in Kotlin

Benutzeroberfläche und App-Architektur (GUI)

Obwohl das Assistenzsystem primär auf eine „Zero-UI"-Bedienung über Sprach- und Audiosignale ausgelegt ist, verfügt die Android-Applikation über eine vollwertige grafische Benutzeroberfläche (GUI). Diese dient essentiellen Zwecken: der Ersteinrichtung (Provisioning) durch sehende Begleitpersonen, der Visualisierung von Systemzuständen sowie dem technischen Debugging der Kamera- und Audiostreams während der Entwicklung.

Single-Activity-Pattern und Navigation

Die Architektur der Applikation folgt dem modernen *Single-Activity-Pattern*. Die `MainActivity` fungiert als globaler Lifecycle-Container und hält die Instanz des `AscentaService`. Die eigentlichen Bildschirminhalte sind in modulare `Fragments` unterteilt (`InfoFragment`, `ImageFragment`, `AudioFragment`). Die Navigation zwischen den Ansichten für Bild- und Audio-Debugging erfolgt fließend und speichereffizient über einen `ViewPager2` in Kombination mit einer `BottomNavigationView`. Der Home-Bildschirm liegt dabei als Overlay über dem Pager und kann jederzeit über einen *Floating Action Button* (FAB) aufgerufen werden.

```
class ViewPagerAdapter(fa: FragmentActivity) : FragmentStateAdapter(fa) {
    override fun getItemCount() = 2
    override fun createFragment(idx: Int) = when(idx) {
        0 -> ImageFragment()
        else -> AudioFragment()
    }
}
// Verknuepfung der Bottom-Navigation mit dem ViewPager
bottomNav.setOnItemSelectedListener { item ->
    hideHome()
    when (item.itemId) {
        R.id.nav_image -> viewPager.currentItem = 0
        R.id.nav_audio -> viewPager.currentItem = 1
    }
    true
```

```
}
```

Listing 4.10: Fragment-Management mittels ViewPager2 in der MainActivity

Verbindungsmanagement (InfoFragment)

Das `InfoFragment` stellt den Einstiegspunkt der Applikation dar. Es visualisiert den aktuellen Verbindungsstatus zwischen Smartphone und Brille („Connected“, „Disconnected“, „Scanning...“) sowie den via I2C ausgelesenen Akkustand der Hardware. Über dieses Interface kann der Nutzer den hybriden *Smart Connection*-Prozess manuell anstoßen. Die Aktualisierung der UI erfolgt Event-gesteuert über ein `ServiceCallback`-Interface, welches sicherstellt, dass Hintergrundereignisse des Sockets korrekt auf dem UI-Thread gerendert werden.

Visuelles Debugging und MediaStore-Integration (ImageFragment)

Zur Verifikation des Sichtfeldes der Brille und zur Validierung der Kameraqualität empfängt das `ImageFragment` Bilddatenpakete via TCP. Eintreffende JPEG-Bytearrays werden dekodiert und als Android `Bitmap` in einer `ImageView` (`iv_stream`) gerendert.

Zusätzlich implementiert dieses Fragment eine Funktion zur dauerhaften Speicherung von Umgebungsaufnahmen. Um modernen Android-Sicherheitsrichtlinien (Scoped Storage ab Android 10) gerecht zu werden, erfolgt die Dateispeicherung nicht über direkte Dateipfade, sondern über die `MediaStore`-API des Betriebssystems. Dies ermöglicht das Ablegen der Bilder in der öffentlichen Galerie des Nutzers, ohne weitreichende Dateisystemberechtigungen anfordern zu müssen:

```
val contentValues = ContentValues().apply {
    put(MediaStore.MediaColumns.DISPLAY_NAME, "Ascenta_${System.
        currentTimeMillis()}.jpg")
    put(MediaStore.MediaColumns.MIME_TYPE, "image/jpeg")
    put(MediaStore.MediaColumns.RELATIVE_PATH, Environment.DIRECTORY_PICTURES +
        "/Ascenta")
}
val uri = resolver.insert(MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
    contentValues)
uri?.let {
    resolver.openOutputStream(it)?.use { stream ->
        bitmap.compress(Bitmap.CompressFormat.JPEG, 100, stream)
    }
}
```

Listing 4.11: Sichere Bildspeicherung über die Android MediaStore API

Audio-Transkription und Wiedergabe (AudioFragment)

Das `AudioFragment` dient der Analyse der bidirektionalen Sprachkommunikation. Es bietet eine Live-Transkription (`tv_transcript`), die den erkannten Text der *Vosk Offline Engine* visuell ausgibt („You: [Erkannter Text]“). Dies ist elementar, um die Genauigkeit des akustischen Modells bei verschiedenen Umgebungsgeräuschen evaluieren zu können.

Zudem implementiert das Fragment Steuerelemente zur manuellen Aufzeichnungsanforderung (`sendCommand("RECORD")`) sowie einen lokalen `MediaPlayer`. Letzterer ermöglicht das erneute Abspielen der im Cache (`mic.wav`) gespeicherten RAW-Audiodaten, um Hardware-Artefakte des Brillenmikrofons isoliert vom Spracherkennungsmodell analysieren zu können. Auch das letzte System-Feedback (TTS) kann über einen dedizierten Button manuell wiederholt werden.

4.2 Herausforderungen in der Entwicklung

Die Realisierung eines kompakten, tragbaren Assistenzsystems, welches rechenintensive Machine-Learning-Prozesse mit kontinuierlicher Netzwerkkommunikation vereint, brachte diverse interdisziplinäre Herausforderungen mit sich. Diese erforderten im Projektverlauf iterative Anpassungen an der Hardware-Architektur sowie tiefgreifende Software-Optimierungen.

4.2.1 Mechanische und geometrische Restriktionen

manuel

4.2.2 Energiemanagement im begrenzten Bauraum

fabian

4.2.3 Ressourcenlimits und Betriebssystem-Restriktionen (Software)

Auf softwareseitiger Ebene stellte der extrem begrenzte Arbeitsspeicher (RAM) des Nicla Vision die größte Hürde dar. Frühe Prototypen der Firmware stürzten regelmäßig mit *Out-Of-Memory* (OOM) Exceptions ab, sobald eine Audioaufnahme gestartet wurde, während das Machine-Learning-Modell im Hintergrund im Framebuffer lag. Die Lösung erforderte den Verzicht auf Standard-Audio-Bibliotheken zugunsten eines selbst entwickelten *Circular Buffers*, der Audiodaten über `memoryview`-Zuweisungen speichereffizient fragmentiert.

Auf Seiten der Android-Applikation resultierten die größten Herausforderungen aus den restriktiven Energiesparmaßnahmen moderner Android-Versionen (Doze-Mode). Da die Applikation primär im Hintergrund bei gesperrtem Smartphone-Display operieren muss, blockierte das Betriebssystem standardmäßig den Empfang von UDP-Broadcasts, was den „Zero-Config“-Verbindungsaufbau verhinderte. Erst die Implementierung eines `Foreground Services` in Kombination mit einem expliziten `WifiManager.MulticastLock` zwang die Hardware dazu, eintreffende Netzwerkpakete zuverlässig an die Applikation durchzureichen, wodurch die Systemstabilität letztlich sichergestellt werden konnte.

4.3 Tests und Validierung

[Hier kommen eure Testergebnisse rein: Wie lange hält der Akku in der Praxis? Wie hoch ist die Latenz der Bild-/Audioübertragung? Wie zuverlässig ist die Tap-Detection beim Gehen?]

5 Ausblick (Future Work)

Der aktuelle Prototyp des Assistenzsystems „Ascenta“ belegt die Machbarkeit einer dezentralen, offline-fähigen Smart-Glasses-Architektur. Um das System jedoch von einem funktionalen Prototyp zu einem marktreifen Endkundenprodukt weiterzuentwickeln, sind in zukünftigen Iterationen verschiedene Erweiterungen und Optimierungen in allen Fachbereichen erforderlich.

5.1 Software und Benutzeroberfläche

Die zukünftige Software-Entwicklung muss sich primär auf die vollständige Barrierefreiheit und erweiterte Navigationskonzepte konzentrieren.

5.1.1 Native Zero-UI-Applikation für blinde Nutzer

Die aktuelle Android-Applikation verfügt über eine grafische Benutzeroberfläche (GUI), die essenziell für das Debugging, die Evaluierung und die Ersteinrichtung durch sehende Begleitpersonen ist. In einer finalen Version für den Endanwender muss diese App jedoch als reine Hintergrundanwendung (*Zero-UI*) konzipiert werden.

Das visuelle Interface wird dabei vollständig obsolet. Die Applikation sollte sich nahtlos in native Accessibility-Dienste (wie Android TalkBack) integrieren und den Setup-Prozess (z. B. die Eingabe des WLAN-Passworts) über auditive Menüführung oder NFC-Tags abwickeln. Der Fokus liegt dabei auf maximaler Stabilität des Foreground-Services und einer Reduktion des Batterie- und Speicherverbrauchs des Smartphones, da keine visuellen Elemente gerendert werden müssen.

5.1.2 Kontinuierliches auditives Raum-Feedback (Leitstrahl-Prinzip)

Eine vielversprechende funktionale Erweiterung stellt die Implementierung eines dynamischen, akustischen Suchmodus dar. Sucht der Nutzer nach einem bestimmten Objekt (z. B. einem Stuhl oder einer Tür), könnte das System ein kontinuierliches Audiosignal generieren, das an die Funktion eines Geigerzählers oder eines Einparkhilfesensors angelehnt ist.

Wenn der Nutzer den Kopf schwenkt und das gesuchte Objekt in das Sichtfeld der Kamera gerät, passt der Algorithmus die Frequenz (Tonhöhe) und das Intervall (Pings pro Sekunde)

des Audiosignals in Echtzeit an. Je zentrierter das Objekt im Sichtfeld ist und je geringer die Distanz (gemessen durch den Time-of-Flight-Sensor) wird, desto intensiver wird das akustische Feedback. Dies würde blinden Nutzern eine hochpräzise, intuitive Zielführung ermöglichen, ohne sie mit komplexen, gesprochenen Sätzen zu überlasten.

5.2 Künstliche Intelligenz und Modell-Training

5.2.1 Erweiterte Objekterkennung für Alltagsgegenstände

Derzeit ist das System auf die Erkennung von Personen optimiert. Für eine umfassende Alltagshilfe muss das zugrundeliegende Dataset massiv erweitert werden. Die Firmware und die App-Logik sind bereits so abstrahiert, dass sie beliebige Labels verarbeiten können. Zukünftige Iterationen müssen daher trainiert werden, um eine Vielzahl von Hindernissen und Zielen (z. B. Treppenstufen, Ampeln, Zebrastreifen, Türen, Sitzgelegenheiten oder herannahende Fahrzeuge) zuverlässig zu klassifizieren.

5.2.2 Weiterentwicklung des Machine-Learning-Modells (Edge Impulse)

[Hier fügt Fabian seinen Ausblick zum Modell-Training ein. Mögliche Themen: Nutzung von leistungsstärkeren Modellen als FOMO, Transfer-Learning mit größeren Datensätzen, Hardware-Beschleunigung durch Ethos-U55 NPU (falls anwendbar), oder das Sammeln von spezifischen Blinden-Datensätzen.]

5.3 Hardware und Elektronik

5.3.1 Optimierung der Schaltungstechnik und Sensorik

[Hier fügt Fabian seinen Ausblick zur Elektronik ein. Mögliche Themen: Integration eines dedizierten Audio-Chips für besseres Mikrofon-Handling, Umstieg auf ein Custom-PCB (Printed Circuit Board) anstelle von Entwickler-Boards, Optimierung des Ladereglers (QuickCharge) oder Einsatz von Solid-State-Batterien für mehr Sicherheit.]

5.4 Mechanik und Gehäusedesign

5.4.1 Weiterentwicklung des 3D-Modells und der Ergonomie

[Hier fügt Manuel seinen Ausblick zum Gehäusedesign ein. Mögliche Themen: Nutzung von flexibleren Materialien (TPU/Resin-Druck), Gewichtsreduktion durch dünnere Wandstärken, IP67-Zertifizierung (Wasserdichtigkeit) durch bessere Dichtungen oder ein modulareres Klick-System für austauschbare Akkus.]

6 Verwandte Arbeiten (Related Work)

Die Entwicklung von Assistenzsystemen für sehbeeinträchtigte Personen ist ein etabliertes Forschungs- und Entwicklungsfeld. Um die Positionierung und den Mehrwert des „Ascenta“-Projekts zu verdeutlichen, werden in diesem Kapitel bestehende Lösungsansätze analysiert und dem hier entwickelten System gegenübergestellt. Die existierenden Systeme lassen sich grob in klassische Hilfsmittel, softwarebasierte Smartphone-Lösungen und dedizierte, kommerzielle Wearables unterteilen.

6.1 Klassische Mobilitätshilfen

Das mit Abstand am weitesten verbreitete Hilfsmittel ist der klassische weiße Langstock. Er ermöglicht durch taktiles Abtasten die Erkennung von Bodenhindernissen, Treppenstufen und Bordsteinkanten. **Abgrenzung:** Der Langstock ist eine essenzielle Basisversorgung, weist jedoch einen entscheidenden blinden Fleck auf: Hindernisse auf Rumpf- und Kopfhöhe (z. B. offene Fenster, Verkehrsschilder oder Äste) werden nicht erfasst, was ein erhebliches Verletzungsrisiko birgt. „Ascenta“ versteht sich nicht als Ersatz für den Langstock, sondern als komplementäres System, welches genau diesen toten Winkel durch den Einsatz des Time-of-Flight-Sensors (ToF) abdeckt.

6.2 Smartphone-basierte Applikationen

In den letzten Jahren haben sich Smartphone-Apps etabliert, die die Handykamera zur Umgebungsanalyse nutzen. Prominente Beispiele sind *Seeing AI* (Microsoft) oder *Be My Eyes*. Diese Applikationen können Texte vorlesen, Währungen erkennen oder Videotelefonate mit sehenden Freiwilligen aufbauen. **Abgrenzung:** Der größte Nachteil dieser Lösungen ist mangelnde Ergonomie. Der Nutzer muss das Smartphone permanent in der Hand halten und aktiv auf Objekte richten, was die Hände für die Nutzung des Langstocks oder von Blindenführhunden blockiert. Zudem basieren diese Apps fast ausschließlich auf Cloud-Computing. Dies erfordert eine konstante Breitband-Internetverbindung und wirft datenschutzrechtliche Bedenken auf, da sensible Videoströme der Umgebung an externe Server gesendet werden. „Ascenta“ löst dieses Problem durch den „Hands-free“-Wearable-Ansatz und die strikte, lokale Datenverarbeitung (Offline-Edge-AI).

6.3 Kommerzielle Smart Glasses und Wearables

Auf dem kommerziellen Markt existieren hochpreisige Speziallösungen. Das bekannteste System ist *OrCam MyEye*, ein kompaktes Kameramodul, das magnetisch an herkömmlichen Brillen befestigt wird. Es liest Texte vor und erkennt Gesichter. Ein weiteres Beispiel sind die *Envision Glasses*, welche auf der Google Glass Enterprise Edition 2 basieren und komplexe Szenenbeschreibungen mithilfe von Künstlicher Intelligenz liefern. **Abgrenzung:** Obwohl diese Systeme technologisch hochentwickelt sind, weisen sie massive Hürden in der Zugänglichkeit auf. Die Anschaffungskosten liegen typischerweise zwischen 2.500 und 4.000 Euro, was sie für einen Großteil der Betroffenen – insbesondere in Entwicklungsländern oder ohne vollständige Kostenübernahme durch Krankenkassen – unerschwinglich macht.

6.4 Positionierung des Ascenta-Projekts

Das Projekt „Ascenta“ positioniert sich als Brückenschlag zwischen kostengünstiger Machbarkeit und datenschutzkonformer Hochtechnologie. Durch die „Split-Brain“-Architektur wird die teure und wärmeintensive Rechenhardware (wie sie in den Envision Glasses verbaut ist) durch das ohnehin beim Nutzer vorhandene Smartphone ersetzt.

Im Gegensatz zu den kommerziellen Systemen belaufen sich die Materialkosten für den Ascenta-Prototyp (Arduino Nicla Vision, LiPo-Akkus, 3D-Druck-Filament) auf unter 150 Euro. Gleichzeitig garantiert die lokale Ausführung des FOMO-Modells sowie der Vosk-Spracherkennungs-Engine eine 100-prozentige Offline-Funktionalität, was die Reaktionszeiten (Latenz) minimiert und die Privatsphäre des Nutzers und dessen sozialen Umfelds absolut schützt.

7 Conclusion

8 Project Management

8.1 Planning

Die organisatorische Planung und Umsetzung des Projekts „Ascenta“ basierte auf einer meilensteinorientierten, agilen Vorgehensweise. Zu Beginn des Projekts (Vorbereitungsphase) lag der Fokus auf der grundlegenden organisatorischen Planung, der Definition von Grob- und Feinzielen sowie der Recherche und Beschaffung der elektronischen Komponenten.

Die formelle Strukturierung der Arbeitsschritte und die Einreichung der Projektmeilensteine erfolgten über das schulspezifische ABA-Portal. Um eine strukturierte Arbeitsteilung zwischen den Bereichen Software, Elektronik und 3D-Druck zu gewährleisten, wurde der Projektplan in wöchentliche Sprints (Kalenderwochen, KW) unterteilt. Zur Überwachung des Fortschritts und der individuellen Arbeitslast wurde eine detaillierte Zeiterfassung (Timesheet) mittels Tabellenkalkulation implementiert, welche die geplanten Freizeitstunden dem tatsächlichen Aufwand (Soll/Ist-Vergleich) kontinuierlich gegenüberstellt.

8.2 Evaluation

Die wöchentliche Evaluierung der Tätigkeitsberichte ermöglichte eine präzise Überwachung des Projektfortschritts. Ein Abgleich der geplanten Ressourcen mit dem tatsächlichen Aufwand zeigt insbesondere in der initialen Projektphase eine deutliche Diskrepanz: Während für die Projektdefinition anfänglich 6 Stunden pro Teammitglied veranschlagt wurden, belief sich der tatsächliche Aufwand auf 20 Stunden pro Person. Diese anfängliche Unterschätzung ist typisch für komplexe Hardware-Software-Kombinationen, da grundlegende Architektur-Entscheidungen zeitintensiv sind.

In den darauffolgenden Wochen (beispielsweise in KW 37 und 38) stabilisierte sich die Arbeitslast und entsprach mit durchschnittlich 6 Stunden pro Teammitglied exakt den Planwerten. Die konsequente Dokumentation der wöchentlichen Ist-Zustände – wie etwa der erfolgreiche Test des Spannungsbetriebsbereichs der Akkus oder die Evaluation der Micro-Speech-Software – half maßgeblich dabei, technische Hürden frühzeitig zu identifizieren und den Projektplan dynamisch anzupassen.

8.3 Timesheet

Die nachfolgenden Unterschriften bestätigen die Korrektheit der in der internen Zeiterfassung dokumentierten, individuell erbrachten Arbeitsstunden für die Diplomarbeit.

8.3.1 Liam Vendel

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 10, 2026	Liam Vendel	_____
Ort, Datum		*Unterschrift*

8.3.2 Manuel Zauner

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 10, 2026	Manuel Zauner	_____
Ort, Datum		*Unterschrift*

8.3.3 Fabian Lindlbauer

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 10, 2026	Fabian Lindlbauer	_____
Ort, Datum		*Unterschrift*

Acknowledgements

List of Figures

Listings

4.1 Akquirierung des MulticastLocks und Start des Foreground Services 9
4.2 Asynchroner UDP-Discovery-Listener in Kotlin 9
4.3 Heartbeat-Coroutine zur Überwachung der Verbindungsstabilität 10
4.4 Berechnung von Delta-G zur Erkennung eines Double-Taps auf den Brillenrahmen 10
4.5 Speichereffizientes Chunking zur Vermeidung von Buffer-Overflows 11
4.6 Dynamische Cooldown-Logik in Abhängigkeit der Objektdistanz 11
4.7 Parsing der Heatmap (4D-Tensor) und Fusion mit ToF-Distanzdaten 14
4.8 Register-Level I2C-Auslesung des MAX17262 Fuel-Gauge-ICs 15
4.9 Sicherer Kontextwechsel zwischen IO-Thread und Main-Thread in Kotlin . . . 15
4.10 Fragment-Management mittels ViewPager2 in der MainActivity 16
4.11 Sichere Bildspeicherung über die Android MediaStore API 17

LIAM VENDEL

Gampern, Österreich | +43 664 1851637 |
Liamvendel06@gmail.com
github.com/schlaimwendla

PROFIL

Angehender Elektronik-Absolvent der HTBLA Braunau mit dem Schwerpunkt **Coding & AI**. Spezialisiert auf Backend Engineering und verteilte Systeme. Fundierte Kenntnisse in Java (OOP, Concurrency) und Low-Level Architektur. Begeisterter Linux-Nutzer (Arch/Garuda) mit Fokus auf Clean Code und Performance.

TECHNISCHE SKILLS

Programmierung:	Java (Fortgeschritten), Python, C++, MicroPython, Assembly (Memory Management).
Systeme & Tools:	Linux (CLI Experte), Docker, Git, Spring Boot, REST APIs, Bash Scripting.
KI & Hardware:	Nicla Vision, ESP32, Raspberry Pi, Edge Impulse, Wit.ai (NLP).
Sprachen:	Deutsch (Muttersprache), Englisch (Fließend).

DIPLOMARBEIT: "ASCENTA"

Smarte Assistenzbrille mit Sprachsteuerung *seit 2025*

- Entwicklung einer synchronisierten **UDP/BLE-Kommunikationsstruktur** zwischen Wearable (Nicla Vision) und Mobile App (Android/Kotlin).
- Integration von **Edge AI** zur Latenzoptimierung (Objekterkennung via Edge Impulse).
- Implementierung einer Offline-Sprachsteuerung zur Gewährleistung maximaler Privatsphäre.

PROJEKTE & HIGHLIGHTS

IoT Light Control System *Raspberry Pi / Python*

- Entwicklung eines REST-API Backends zur modularen und drahtlosen Steuerung von Lichtmodulen.

Automatic Guitar Tuner *Arduino / Signalverarbeitung*

- Design eines elektromechanischen Stimmgeräts auf Basis von Frequenzanalyse und Feedback-Loops.

European Space Camp 2025 *Andøya Space Center, Norwegen*

- Auswahl für das internationale Forschungs-Camp (Fokus: Sounding Rockets & Systems Engineering).

AUSBILDUNG

HTBLA Braunau **2021 – 2026**

Höhere Technische Bundeslehranstalt für Elektronik und Technische Informatik
Schwerpunkt: **Coding & AI** (Voraussichtlicher Abschluss: Juni 2026)

INTERESSEN & ENGAGEMENT

- **Open Source:** Linux-Kernel & System-Optimierung (Btrfs-Subvolumes, Kernel-Tuning).
- **Sport:** Aktiver Volleyball-Spieler und Jugend-Coach im Verein.
- **Raumfahrt:** NASA Extracurricular Kurse in Weltraumphysik und orbitaler Mechanik.

Manuel Zauner

Höfter Straße 70a

5280 Braunau am Inn

Tel: 0677 62550900

E-Mail: manuel.zauner.2000@gmail.com

Lebenslauf

Angaben zur Person

Name:	Manuel Zauner
Geburtsdatum:	19.01.2007
Geburtsort:	Braunau am Inn
Staatsbürgerschaft:	Österreich

Familiensituation:

Vater:	Christian Zauner (Technischer Angestellter, Fa. Spiratec)
Mutter:	Bettina Zauner (Personalverrechnung, Fa. EKB)
Geschwister:	Thomas Zauner (Gymnasium Braunau)

Schulbildung

Art/Dauer der Schule:	2013 – 2017	VS Braunau-Laab
	2017 – 2021	Gymnasium Braunau
	2021 – laufend	HTL Braunau (Coding & AI)
Lieblingsfach:	PBE_3 (Prototypenbau elektronischer Systeme)	

Berufserfahrung

Ferialjob:	August 2023	4 Wochen in der Firma Dräxlmaier
	August 2024	4 Wochen in der Firma Dräxlmaier

Sonstige Angaben

Besondere Fähigkeiten:	EDV, gute Menschenkenntnisse
Hobbys:	Basketball, Krafttraining, Tennis, Computer

Lebenslauf

Name: Fabian Lindlbauer
Geburtsdaten: 24.07.2007
Anschrift: Moritzhub 8, 4950 Altheim
Telefon: 06504307493
E-Mail: fabian.lindlbauer007@gmail.com

Schulausbildung:

2013-2017	Volksschule Altheim
2017-2021	Mittelsschule Altheim
2021-	HTL Braunau

Berufserfahrung:

Juli-Juli 2023	Hargassner Weng	Ferialpraktikum
Juli-Juli 2024	SGS Dorf an der Pram	Ferialpraktikum
Juli-Juli 2025	HTL Braunau Schulpartnerschaft Nicaragua	

Fremdsprache:

Englisch B2

ECDL
Führeschein B

Altheim, 11.09.2025

Fabian Lindllbauer